

1 UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
2 SCUOLA POLITECNICA E DELLE SCIENZE DI BASE
3 DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
4 DELL'INFORMAZIONE
5 CORSO DI LAUREA TRIENNALE IN INFORMATICA

6 A FANCY L^AT_EX TEMPLATE FOR YOUR THESIS
7 AT UNINA

8 **Relatori**

Megadir. Prof. Guidobaldo Maria
RICCARDELLI
Duca Conte Francesco BARAMBANI
Baronessa Filiguelli DE BONCHAMPS

Correlatore

Megadir. Clam. Duca Conte Pier
Carlo ing. SEMENZARA

Candidato

Marco FESTA
N86004979

Sommario

1

2 Il presente lavoro di tesi illustra le attività di ricerca e sviluppo condotte durante il periodo
3 di tirocinio presso **InfoCube**, focalizzandosi sull'estrazione, strutturazione e interroga-
4 zione semantica avanzata di documenti legali e normativi. Il progetto si inserisce nel
5 contesto più ampio di **LUDOVICA** (*Language Understanding for Document Optimization*
6 *and Validation In Civil Administration*), un motore generativo basato sull'Intelligenza
7 Artificiale volto a supportare la Pubblica Amministrazione Locale nell'elaborazione e
8 stesura di atti amministrativi.

9 Il problema tecnico principale affrontato in questo lavoro è la trasformazione di do-
10 cumenti PDF complessi in una base di conoscenza solida e interrogabile dalle macchine
11 (**Knowledge Graph**). A tale scopo, la rigida gerarchia tipica degli atti normativi (Tito-
12 li, Capi, Articoli, Commi) è stata modellata e salvata all'interno del database a grafo
13 **Neo4j**, permettendo di preservare fedelmente le relazioni strutturali e concettuali dei testi
14 originali.

15 Per abilitare il recupero intelligente e contestuale delle informazioni, è stato imple-
16 mentato un sistema di **Semantic Search**. Questa fase ha richiesto il testing di molteplici
17 **modelli di embedding**, allo scopo di individuare la soluzione più performante per la
18 vettorializzazione del complesso gergo giuridico italiano. Al fine di validare le scelte
19 architetturali, è stato inoltre introdotto e impiegato il database vettoriale **Qdrant** a scopo
20 puramente sperimentale. Ciò ha permesso di condurre un'analisi comparativa diretta tra
21 le performance di un approccio di ricerca puramente vettoriale e i vantaggi offerti dalla
22 navigazione topologica nel grafo.

23 SCRIVERE QUI I RISULTATI OTTENUTI, LE CONCLUSIONI E EVENTUALI PRO-
24 SPETTIVE FUTURE.

Indice

2	1	Introduzione	1
3	1.1	Definizione del problema	1
4	1.2	Obiettivi del lavoro di tesi	2
5	1.3	Tecnologie utilizzate	2
6	2	Architettura del Sistema	4
7	2.1	Componenti del sistema	4
8	3	Implementazione Software	6
9	3.1	Estrazione dei Documenti	6
10	3.1.1	Open Data Loader e la Gestione dei Tagged PDF	6
11	3.1.2	Docling e l'Integrazione dei Motori OCR	7
12	3.1.3	Progettazione del Custom Triage Processor	8
13	3.2	Strutturazione dei Documenti tramite LLM	11
14	3.2.1	Il Modello e la Strategia di Generazione	11
15	3.2.2	Generazione della Struttura Gerarchica	11
16	3.2.3	Estrazione delle Relazioni Semantiche	12
17	3.3	Importazione dati nei Database	13
18	3.3.1	I Modelli di Dominio	13
19	3.3.2	Disaccoppiamento della Persistenza: Lo Strategy Pattern	14
20	3.3.3	Modalità di Accesso ai Dati: Il Repository Pattern	14
21	3.3.4	Dettagli Implementativi dell'Ingestion su Neo4j	14
22	3.3.5	Dettagli Implementativi dell'Ingestion su Qdrant	15
23	3.4	Creazione degli Embedding	16
24	3.4.1	Astrazione e Gestione dei Modelli di Embedding	16
25	3.4.2	Integrazione con i Database	17

-1-

Introduzione

CONTENTS: 1.1 Definizione del problema. 1.2 Obiettivi del lavoro di tesi. 1.3 Tecnologie utilizzate.

1.1 Definizione del problema

La Pubblica Amministrazione Locale italiana fa quotidianamente affidamento su una vasta e complessa mole di testi normativi, atti e delibere. L'elaborazione automatica di tali documenti è tuttavia ostacolata da diverse sfide intrinseche: i testi sono spesso archiviati in formati non strutturati (come i PDF), presentano difformità di impaginazione, impiegano un linguaggio burocratico peculiare e si articolano in una rigida e profonda struttura gerarchica. Quando si tenta di automatizzare la ricerca e la stesura di nuovi atti attraverso sistemi basati su Intelligenza Artificiale Generativa, l'approccio classico - basato unicamente sul recupero esplorativo di porzioni di testo destrutturato (frammenti di testo o "chunk") - si rivela insufficiente, portando inevitabilmente alla potenziale perdita del contesto legale e allo smarrimento delle relazioni gerarchiche originarie dell'atto documentale.

È in questo scenario che si inserisce il lavoro di tesi, svolto nell'ambito delle attività di tirocinio presso **InfoCube** per contribuire allo sviluppo del progetto **LUDOVICA**, il cui obiettivo finale è fornire ai funzionari della PA un motore generativo affidabile, capace di recuperare gli atti pregressi pertinenti e generare template documentali pronti all'uso. Affinché le risposte del sistema siano rigorose e verificabili è emersa l'esigenza di adottare un'architettura dati che introduca una solida topologia informativa.

Il problema tecnico fondamentale affrontato consiste quindi nella necessità di **convertire i documenti normativi complessi in una base di conoscenza interrogabile** (*Knowledge Graph*) che conservi fedelmente la struttura semantica e gerarchica originale (Titoli, Capi, Articoli).

1.2 Obiettivi del lavoro di tesi

Al fine di superare i limiti esposti e dotare il sistema di solidi meccanismi per l'interpretazione del linguaggio giuridico, il lavoro si è articolato nei seguenti obiettivi:

1. **Modellazione topologica e digitalizzazione:** Costruire una pipeline di inserimento per tradurre documenti legali non strutturati in un database a grafo (*Neo4j*), definendo rigorosamente la mappatura delle relazioni che legano tra loro le sezioni della gerarchia normativa.
2. **Semantic Search per il dominio giuridico:** Sviluppare il modulo di interrogazione intelligente del database e quantificare, mediante il testing di molteplici modelli di embedding, quale sia la soluzione più performante nel catturare la semantica intrinseca del linguaggio legale in lingua italiana.
3. **Analisi prestazionale degli approcci di Retrieval:** Integrare un database puramente vettoriale (*Qdrant*) per poter eseguire un **confronto diretto**. L'obiettivo è misurare la differenza qualitativa delle risposte e confrontare le prestazioni fornite da un approccio di ricerca puramente vettoriale, rispetto a un'interrogazione che sfrutti anche la navigazione topologica sul grafo.

1.3 Tecnologie utilizzate

Per far fronte alle sfide di estrazione, elaborazione, archiviazione e interrogazione dei documenti normativi, è stato impiegato uno stack tecnologico che combina strumenti di parsing avanzato, modelli di LLM ed embedding e database specializzati:

Linguaggi di Programmazione

- **Python:** Scelto per la vastità e la potenza delle sue librerie, è stato il linguaggio cardine per la costruzione delle pipeline del progetto.

Strumenti per l'Estrazione del Testo

- **Open Data Loader:** Una libreria open-source scelta per la sua capacità di estrarre testo in modo *deterministico* da file PDF, convertendolo direttamente in Markdown.
- **Docling:** Una libreria open-source di parsing e analisi del layout documentale, specializzato nell'estrazione precisa di testo, tabelle e metadati strutturati a partire da formati complessi come i PDF. Integrato come supporto di Open Data Loader.
- **RapidOCR:** Modulo impiegato per estrarre testo dai documenti derivanti da scansioni visive. Tra i vari motori OCR disponibili, è stato optato l'uso di questo specifico

1 OCR che si è rivelato il compromesso più efficace per bilanciare i tempi di esecuzione
2 e l'elevato consumo di RAM del container.

3 Database e Linguaggi di Interrogazione

- 4 • **Neo4j:** Il database a grafo principale utilizzato per mappare la rigida struttura
5 gerarchica degli atti normativi e i nodi concettuali. Consente di gestire e navigare le
6 relazioni in maniera altamente efficiente rispetto ai database relazionali tradizionali.
- 7 • **Cypher:** Il linguaggio di interrogazione dichiarativo nativo di Neo4j, utilizzato per
8 effettuare le query di inserimento (ingestion) e di ricerca topologica nel grafo.
- 9 • **Qdrant:** Un database vettoriale utilizzato sperimentalmente per condurre test e
10 confronti sulle performance della ricerca semantica (*Cosine Similarity*) rispetto
11 all'approccio basato sui grafi.

12 LLM e Modelli di Embedding

- 13 • **Gemini 2.5-flash:** *Large Language Model* utilizzato tramite API per il compito
14 cruciale della strutturazione sintattica. Riceveva in input il testo in formato Mar-
15 kdown dai PDF e restituiva JSON formattati rigorosamente secondo uno schema
16 predefinito, che rappresentavano la gerarchia dei documenti normativi (Titoli, Capi,
17 Articoli, Commi) e i nodi concettuali.
- 18 • **Modelli di Embedding:** Sono stati testati svariati modelli per ottimizzare la vet-
19 torializzazione del testo legale in italiano. I test più approfonditi hanno coinvolto i se-
20 guenti modelli: `intfloat/multilingual-e5-large`, `google/gemini-embedding-001`
21 e `BAAI/bge-m3`. La valutazione si è basata sulla capacità di recuperare commi
22 pertinenti a partire da query in linguaggio naturale.

23 Deployment e Containerizzazione

- 24 • **Docker:** L'intero ecosistema è stato modularizzato attraverso lo sviluppo di contai-
25 ner isolati e specifici. Questa scelta ha garantito un'elevata modularità, l'isolamento
26 delle dipendenze software e un rigoroso controllo sulle risorse computazionali, spe-
27 cialmente durante i task più onerosi legati all'*Optical Character Recognition (OCR)* e
28 alla *generazione degli Embedding*.

-2-

Architettura del Sistema

3 CONTENTS: 2.1 Componenti del sistema.

4 In questo capitolo viene presentata l'architettura complessiva del sistema sviluppato,
5 illustrando i componenti principali e il flusso di dati attraverso le diverse fasi del processo.
6 L'obiettivo è fornire una panoramica chiara e dettagliata delle scelte progettuali adottate
7 per affrontare le sfide tecniche del progetto.

8 2.1 Componenti del sistema

9 I componenti sono presentati in ordine logico, rappresentando il percorso che i dati
10 seguono dal momento in cui vengono acquisiti fino alla fase di interrogazione e utilizzo
11 da parte degli utenti finali.

12 1. **Estrazione dei Documenti:** Descrizione delle tecniche utilizzate per l'estrazione
13 dei testi dai documenti normativi, con particolare attenzione alle strategie e alle
14 librerie impiegate per il parsing e la pulizia dei dati.

15 2. **Strutturazione dei Documenti:** Spiegazione del ruolo dei LLM nella trasfor-
16 mazione del testo estratto in un formato strutturato, con particolare attenzione
17 alla definizione dello schema JSON utilizzato per rappresentare la gerarchia dei
18 documenti normativi.

19 3. **Archiviazione nei Database:** Descrizione delle tecnologie di storage utilizzate,
20 con focus sui due principali database impiegati:

21 3.1 **Database a Grafo (Neo4j):** Illustrazione della struttura del database e del
22 processo di inserimento dei dati, con particolare attenzione alla modellazio-
23 ne delle informazioni e alla rappresentazione delle relazioni gerarchiche dei
24 documenti normativi.

- 1 **3.2 Database Vettoriale (Qdrant):** Descrizione del processo di inserimento dei
2 dati strutturati nel database
- 3 **4. Implementazione della Ricerca Semantica:** Spiegazione dell'approccio adottato
4 per la ricerca semantica, compresa la scelta dei modelli di embedding e le definizioni
5 delle metriche di valutazione.
- 6 **5. Benchmarking e Confronto tra Database:** Processo di benchmark condotto per
7 valutare le prestazioni di ricerca, analizzando le differenze e le performance tra
8 l'approccio basato su database a grafo e quello basato su database vettoriale.
- 9 La figura 2.1 fornisce un diagramma riassuntivo dell'architettura del sistema, evidenziando
10 i componenti principali e il flusso di dati tra di essi.

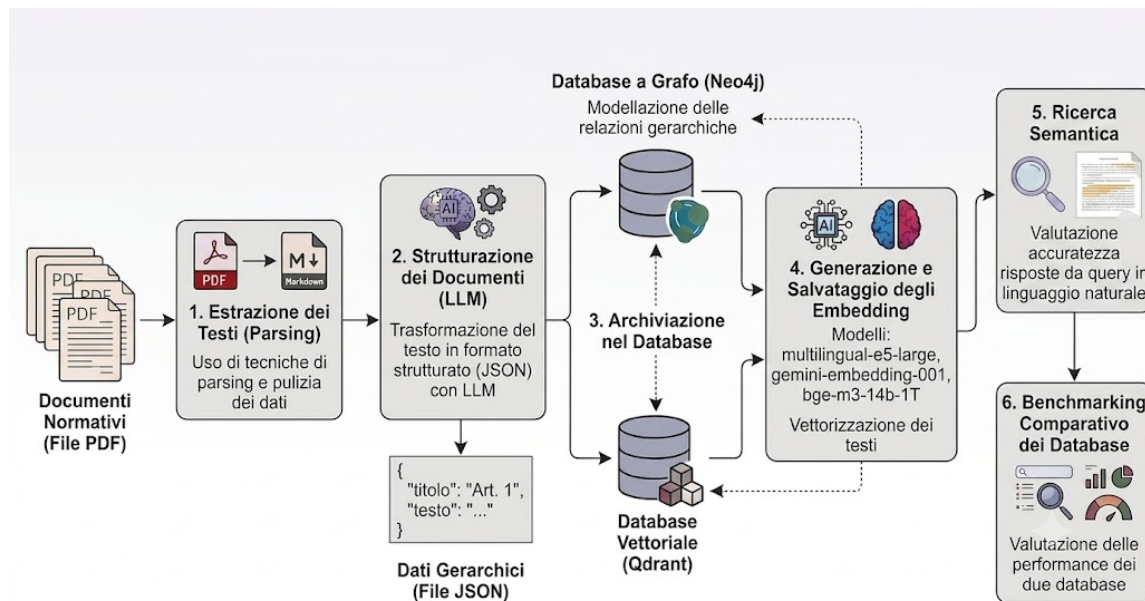


Figura 2.1: Architettura del sistema

–3–

Implementazione Software

CONTENTS: 3.1 Estrazione dei Documenti. 3.2 Strutturazione dei Documenti tramite LLM. 3.3 Importazione dati nei Database. 3.4 Creazione degli Embedding.

In questo capitolo verranno analizzate nel dettaglio le singole fasi della pipeline di elaborazione, partendo dalle strategie di estrazione del testo grezzo, passando per la strutturazione semantica operata dai Large Language Models, fino ad arrivare alle logiche di ingestione e interrogazione sui database

3.1 Estrazione dei Documenti

La fase iniziale della pipeline prevede l'estrazione del testo dai documenti normativi in formato PDF.

Nelle prime fasi di sviluppo del progetto, l'intero documento PDF veniva passato direttamente al Large Language Model (Gemini), demandando a quest'ultimo sia il compito di estrazione del testo grezzo sia quello di strutturazione sintattica. Questo approccio presentava forti criticità, esponeva il sistema al rischio di allucinazioni ed era inefficiente nel parsing di strutture complesse. Per superare tali limitazioni, la pipeline è stata modificata separando nettamente l'estrazione del testo dalla sua successiva strutturazione.

3.1.1 Open Data Loader e la Gestione dei Tagged PDF

Per l'estrazione del testo da documenti digitali nativi è stata adottata la libreria open-source *Open Data Loader*. Questo strumento consente di convertire localmente i file PDF in formati pronti per l'alimentazione degli LLM, specificamente Markdown e JSON, garantendo un'accurata ricostruzione dell'ordine di lettura e un'efficace estrazione delle tabelle e dei relativi bounding box associati agli elementi grafici. Il vantaggio di questa soluzione risiede nella sua natura puramente deterministica: a parità di input viene

1 garantito lo stesso identico output, azzerando le variabilità tipiche dei modelli generativi
2 in fase di pura lettura del testo.

3 Una delle principali problematiche riscontrate nell'estrazione da documenti legali
4 riguarda la presenza di a capo spuri all'interno del file PDF originale, i quali, se non gestiti,
5 introducono doppi ritorni a capo (\n\n) nel testo Markdown, frammentando artificialmen-
6 te i periodi e inficiando la qualità della successiva fase di chunking. Per risolvere questa
7 criticità, all'interno dello script di estrazione è stato configurato il parametro specifico:
8 `use_struct_tree=True`.

9 Quando questo parametro è abilitato, la libreria verifica se il file analizzato appartiene
10 alla categoria dei *Tagged PDF*. A differenza dei PDF standard, che si limitano a definire
11 il layout visivo e geometrico della pagina, i Tagged PDF incorporano un livello logico
12 aggiuntivo e invisibile all'utente: il *Logical Structure Tree* (Albero della Struttura Logica).
13 Questo albero gerarchico è composto da elementi strutturali (*Structural Element Nodes*)
14 paragonabili ai tag del linguaggio HTML (quali <H1>, <P>, <Table>, <L> per le liste).
15 Ogni porzione di testo visibile sulla pagina è mappata in modo univoco all'interno di uno
16 di questi nodi, specificandone esplicitamente il ruolo semantico e l'esatto ordine logico
17 di lettura, indipendentemente dalla disposizione grafica dei blocchi di testo nello spazio
18 bidimensionale.

19 L'abilitazione di `use_struct_tree=True` permette quindi a Open Data Loader di
20 ignorare completamente le euristiche visive o geometriche di estrazione — che spesso falli-
21 scono o introducono rumore — e di navigare direttamente lo *Structure Tree* del documento.
22 Questa tecnica consente di discriminare con precisione assoluta gli a capo puramente
23 stilistici (dovuti ad esempio dal raggiungimento del margine della pagina) dalle effettive
24 interruzioni di paragrafo, restituendo un testo continuo, pulito e semanticamente fedele
25 all'originale.

26 Nel contesto specifico di questo lavoro di tesi, focalizzato sui documenti normativi
27 e sugli atti amministrativi della Pubblica Amministrazione Locale, l'approccio basato
28 sui Tagged PDF si è rivelato straordinariamente efficace per ragioni sia pratiche che
29 legali. Le istituzioni pubbliche sono infatti vincolate da stringenti obblighi normativi in
30 materia di accessibilità digitale. Tali norme impongono che ogni documento pubblicato
31 sui portali istituzionali sia pienamente fruibile anche da utenti con disabilità visive o
32 motorie attraverso l'uso di tecnologie assistive, come gli screen reader.

33 La conformità a questi standard garantisce che il *Logical Structure Tree* del PDF non
34 solo sia presente, ma risulti strutturato accuratamente: i testi seguono una gerarchia
35 rigida, le tabelle possiedono intestazioni chiare per righe e colonne, e l'ordine logico di
36 lettura è validato direttamente alla fonte. Di conseguenza, quello che per l'ente pubblico
37 rappresenta un adempimento di legge obbligatorio, si traduce in una vera e propria “ancora
38 di salvezza”.

39 3.1.2 Docling e l'Integrazione dei Motori OCR

40 L'approccio standard basato su Open Data Loader mostra evidenti limiti in presenza di
41 documenti non strutturati, privi di testo selezionabile o derivanti da scansioni visive,

in quanto la libreria non integra nativamente un modulo OCR. Per sopperire a questa mancanza e gestire layout documentali complessi, è stata introdotta nel sistema la libreria *Docling*, un motore avanzato di parsing e analisi del layout specializzato nell'estrazione di metadati, tabelle e testo da formati complessi, integrato nella pipeline come supporto a Open Data Loader.

L'introduzione dell'OCR ha sollevato complessità non trascurabili relative alla gestione delle risorse hardware all'interno dell'architettura a container. Il container Docker deputato all'estrazione operava infatti con un vincolo stringente di memoria RAM pari a un massimo di 4 GB, per questioni di risorse hardware. Durante i primi test, l'invio massivo di interi documenti scansionati causava la saturazione istantanea della memoria, provocando l'intervento del modulo del kernel Linux (*Out-Of-Memory Killer*) che terminava forzatamente il container.

Per superare il problema del consumo di RAM e prevenire i crash di sistema, è stata implementata una strategia basata sull'elaborazione pagina-per-pagina e si è proceduto a un'attività di comparazione su tre diversi motori OCR supportati da Docling:

1. **EasyOCR:** Configurato di default all'interno di Docling, adotta modelli basati su Deep Learning altamente accurati ma estremamente esigenti in termini computazionali. Anche isolando l'elaborazione sulla singola pagina, il consumo di RAM superava costantemente la soglia critica dei 4 GB, rendendone impossibile l'utilizzo nell'ambiente di produzione containerizzato.
2. **Tesseract OCR:** Si è dimostrato estremamente leggero e pienamente compatibile con i limiti di memoria imposti, scongiurando qualsiasi crash del container. Tuttavia, nel contesto specifico del linguaggio legale italiano, il motore non è stato in grado di interpretare correttamente il layout, fallendo l'estrazione del testo e restituendo all'interno del file Markdown unicamente i link di riferimento alle immagini scansionate.
3. **RapidOCR:** Questo specifico modulo è stato infine selezionato per l'estrazione del testo dalle scansioni visive. RapidOCR si è rivelato il compromesso ingegneristico ideale, offrendo un perfetto bilanciamento tra l'accuratezza del riconoscimento del testo italiano e il consumo di RAM del container, che si è mantenuto costantemente sotto la soglia di guardia. A fronte di un tempo di computazione non trascurabile — quantificabile in circa 18-20 secondi per singola pagina scansionata — questo motore assicura la totale stabilità infrastrutturale dell'intera pipeline.

3.1.3 Progettazione del Custom Triage Processor

Il processore di triage nativo di Open Data Loader prevede un flusso standard in cui ogni pagina viene analizzata e smistata: le pagine semplici vengono processate tramite il processo di Open Data Loader (una JVM), mentre le pagine complesse vengono delegate al backend basato su Docling.

1 Tuttavia, l'applicazione con l'uso del parametro `use_struct_tree=True` introduce
2 un bug logico nel triage di default: i documenti non espressamente *tagged* vengono
3 forzati all'interno del processo standard di Open Data Loader, non venendo mai instradati
4 verso il backend di Docling, escludendo così l'analisi per pagine troppo complesse oppure
5 l'attivazione dell'OCR dove necessario. Per risolvere questa problematica è stato progettato
6 e implementato un **Custom Triage Processor**. Il componente agisce secondo la seguente
7 logica di controllo:

- 8 • **Fase 1: Verifica preliminare della struttura.** Non appena il file PDF viene ac-
9 quisito, il sistema ne analizza i metadati per verificare se è codificato come *Tagged*
10 *PDF*. In caso affermativo, l'intero documento viene elaborato direttamente da Open
11 Data Loader sfruttando l'albero strutturale nativo. Questo approccio garantisce la
12 massima velocità di esecuzione e una pulizia ottimale del testo estratto.
- 13 • **Fase 2: Frazionamento del documento e stima della complessità.** Se il do-
14 cumento risulta privo di struttura (*Non-Tagged*), viene diviso in singole pagine
15 indipendenti. Questa separazione è fondamentale per prevenire sovraccarichi di
16 memoria RAM durante le elaborazioni successive. Per ciascuna pagina, il sistema
17 calcola poi la densità dei caratteri testuali per valutarne il grado di complessità.
- 18 • **Fase 3: Triage ed elaborazione mirata della pagina.** Sulla base dell'analisi
19 precedente, ogni singola pagina subisce un processo di smistamento intelligente
20 per essere indirizzata allo strumento di estrazione più adeguato:
 - 21 – Se la pagina presenta un layout lineare e un'elevata densità di testo nativo,
22 viene classificata come *Semplice* e affidata al processo di Open Data Loader.
 - 23 – Se la pagina contiene elementi geometrici articolati o tabelle annidate, ma
24 il testo rimane comunque selezionabile, viene classificata come *Complessa* e
25 processata tramite *Docling Standard* (senza abilitare l'OCR).
 - 26 – Se la pagina è priva di testo nativo (es. una fotocopia), viene classificata come
27 *Scansione* e inviata al container specializzato di *Docling*, dove interviene il
28 motore *RapidOCR* per il riconoscimento ottico dei caratteri.

29 L'intero processo appena descritto è schematizzato nella Figura 3.1.

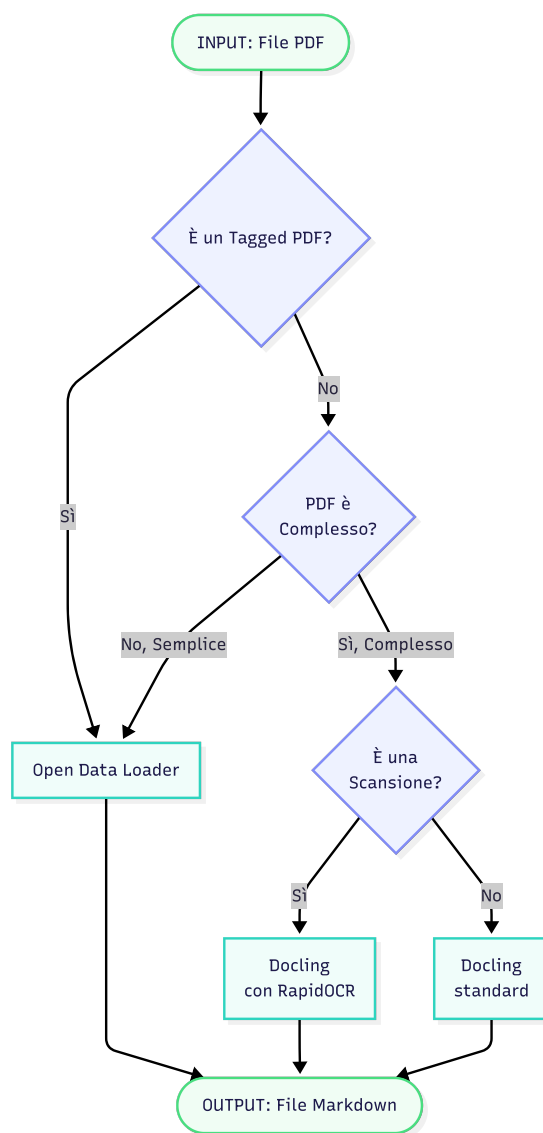


Figura 3.1: Flowchart del Custom Triage Processor

1 Grazie a questa architettura a triage personalizzata, il sistema ha dimostrato una
2 robustezza assoluta durante la fase di testing. Ad esempio, analizzando un batch di 11
3 documenti eterogenei estratti dalla Provincia di Taranto — composto nello specifico da
4 6 documenti nativamente *tagged*, 3 documenti *non-tagged* a testo nativo e 2 documenti
5 interamente scansionati — il sistema ha completato l'intero processo di estrazione in
6 1013.33 secondi (circa 17 minuti), senza registrare alcuna perdita di dati o fallimenti
7 catastrofici dovuti alla memoria.

8 NOTE: Aggiungere statistiche con i risultati più recenti ottenuti. Includere: il numero
9 totale di pagine elaborate, la distribuzione tra pagine semplici, complesse e scansionate, e
10 il tempo medio di elaborazione per ciascuna categoria. (Ora sto a 33 documenti elaborati)

1 3.2 Strutturazione dei Documenti tramite LLM

2 Una volta ultimata la fase di estrazione del testo grezzo, illustrata nella sezione precedente,
3 i documenti normativi si presentano sotto forma di file testuali piatti in formato Mark-
4 down. Tali file, pur contenendo l'intero testo originale, sono del tutto privi di metadati
5 strutturali e di una formattazione gerarchica chiara. Poiché l'architettura di sistema
6 prevede l'ingestione di questi documenti all'interno di un database a grafo (Neo4j) e in un
7 database vettoriale (Qdrant), risulta indispensabile trasformare il testo non strutturato
8 in una gerarchia rigorosa. L'archiviazione del solo testo grezzo, infatti, risulterebbe del
9 tutto inadeguata e priva di utilità pratica. Una strutturazione accurata rappresenta il pre-
10 requisito fondamentale per abilitare qualsiasi operazione avanzata di *information retrieval*,
11 garantendo che i dati possano essere interrogati in modo efficiente e contestuale.

12 3.2.1 Il Modello e la Strategia di Generazione

13 Per affrontare il compito cruciale della strutturazione sintattica, è stato impiegato il Large
14 Language Model Gemini 2.5-flash. A livello di architettura software, il processo è
15 diviso in due step distinti: la generazione di una struttura JSON gerarchica e l'estrazione
16 delle relazioni semantiche tra le entità legali.

17 3.2.2 Generazione della Struttura Gerarchica

18 L'efficacia e l'accuratezza del processo di strutturazione si fondano su un *System Prompt* in-
19 gegnerizzato ad hoc. Il suo compito principale è quello di mappare il contenuto all'interno
20 del seguente schema logico:

- 21 • **Metadati (metadata):** Il modello estrae e categorizza i dati identificativi dell'atto.
22 Vengono isolati campi chiave come la data di stipula, il titolo completo e il nome
23 del file originale.
- 24 • **Sezioni Introduttive e Conclusive:** Vengono separate dal corpo normativo tut-
25 te le sezioni di contorno. Tra queste figurano le premesse iniziali e le epigrafi
26 (preface), gli indici (preamble), le dichiarazioni di chiusura con le relative firme
27 (conclusions) e gli eventuali documenti annessi (attachments).
- 28 • **Corpo del Documento (body):** Rappresenta il nucleo dell'algoritmo di struttura-
29 zione. Il modello deve nidificare il testo seguendo fedelmente la gerarchia complessa
30 e rigida tipica degli atti normativi italiani: **Titoli > Capi > Articoli > Commi**.

31 Per garantire la massima coerenza dei dati all'interno del Knowledge Graph, sono
32 state imposte al LLM regole tassative di parsing:

- 33 • **Normalizzazione Numerica:** Tutti gli identificativi gerarchici originariamente
34 espressi in numeri romani o in forma testuale (es. "TITOLO I", "CAPO IV") devono
35 essere categoricamente convertiti in numeri interi arabi (1, 4, ecc.).

- 1 • **Gestione Dinamica dei Commi:** Poiché nei testi di legge i commi non sono
2 sempre esplicitamente numerati, il modello è istruito a dedurli analizzando le
3 interruzioni logiche dei paragrafi, assegnando loro una numerazione progressiva.
 - 4 • **Ottimizzazione dello Schema:** Per mantenere leggeri i file JSON, il modello è
5 programmato per non inizializzare chiavi con valori vuoti o nulli; se un livello
6 gerarchico (come un Capo o un Titolo) non trova riscontro nel testo originale, la
7 proprietà viene semplicemente omessa.
- 8 L'esecuzione di questo processo restituisce una directory di documenti JSON puliti e
9 strutturati gerarchicamente, pronti per essere archiviati e per affrontare il successivo step:
10 l'estrazione delle relazioni semantiche tra le entità legali.

11 3.2.3 Estrazione delle Relazioni Semantiche

12 La sola scomposizione del testo normativo in una struttura gerarchica ad albero rap-
13 presenta una condizione necessaria ma non sufficiente per la costruzione di un vero e
14 proprio *Knowledge Graph*. Per sfruttare appieno la potenza topologica di un database a
15 grafo come Neo4j, è indispensabile individuare e mappare esplicitamente i collegamenti e
16 le dipendenze logiche che intercorrono tra le diverse disposizioni giuridiche all'interno
17 dello stesso atto. A questo scopo, la pipeline prevede un secondo step logico interamente
18 dedicato all'estrazione delle relazioni semantiche. In questa fase, l'input non è più il testo
19 grezzo, ma i file JSON prodotti nello step precedente, che quindi agiscono come una base
20 di conoscenza consolidata. Ciascun documento viene quindi analizzato dal LLM Gemini
21 2.5-flash, guidato da un *System Prompt* specializzato. Tale prompt è stato progettato
22 per riconoscere i costrutti linguistici del gergo giuridico che indicano un collegamento
23 formale tra le parti del testo. Le relazioni estratte sono:

- 24 • **CITA:** Un articolo o comma fa riferimento a un'altra disposizione.
- 25 • **ABROGA:** Una disposizione annulla la validità giuridica di un'altra.
- 26 • **DEROGA:** Un elemento introduce un'eccezione a una regola generale.
- 27 • **SOSTITUISCE:** Una disposizione prende il posto di un'altra.
- 28 • **INTERPRETA:** La comprensione di una norma dipende dalla lettura di un'altra.

29 L'output di questa analisi è una rete di collegamenti formali, dove per ciascuna rela-
30 zione vengono specificati il nodo di partenza, il nodo di arrivo e il tipo di legame. Queste
31 informazioni sono esportate in un nuovo set di file JSON.

32 Questo approccio a due stadi permette di separare le responsabilità e ridurre il carico
33 cognitivo del modello a ogni ciclo. In questo modo si produce un dataset pulito e affidabile,
34 pronto per essere caricato nel grafo.

1 3.3 Importazione dati nei Database

2 Una volta completata la strutturazione dei documenti legali in formato JSON, si apre la
3 fase di persistenza e indicizzazione dei dati all'interno dei database. Nel contesto di questo
4 progetto di tirocinio, l'architettura prevede il caricamento dei dati estratti all'interno
5 di due distinte tipologie di database: Neo4j, un database a grafo, e Qdrant, un database
6 vettoriale.

7 L'uso di due database consente di sperimentare l'uso di entrambi i paradigmi di
8 memorizzazione sullo stesso dominio normativo, al fine di valutarne i rispettivi punti di
9 forza, analizzare le performance e far emergere differenze qualitative e quantitative nelle
10 fasi di interrogazione e navigazione dei testi legali.

11 Per garantire la coesistenza flessibile di queste due tecnologie e favorire un ambiente
12 di sviluppo pulito e modulare, lo sviluppo del software ha visto l'applicazione combinata
13 di due noti design pattern software: lo *Strategy Pattern* e il *Repository Pattern*.

14 3.3.1 I Modelli di Dominio

15 Prima di analizzare le strategie di persistenza, è fondamentale descrivere come i dati
16 estratti vengano rappresentati in memoria. All'interno della cartella `models` di questo
17 step, il sistema definisce le entità fondamentali del dominio normativo utilizzando il
18 paradigma ad oggetti. Queste classi operano essenzialmente come *Entities*: strutture dati
19 pure, prive di logica di persistenza diretta o dipendenze verso i DBMS, il cui unico scopo
20 è incapsulare i dati e preservare i vincoli di integrità del dominio.

21 L'architettura dei modelli rispecchia fedelmente la scomposizione gerarchica del testo
22 legale:

- 23 • **Entity**: Rappresenta l'ente pubblico emittente (ad esempio, un Ministero o un
24 Comune), memorizzando i metadati identificativi dell'istituzione.
- 25 • **LegalAct**: Modella l'atto normativo nella sua interezza. Al suo interno, oltre ai
26 metadati generali del documento, espone il metodo `from_json_file(file_path)`,
27 delegato al parsing del file JSON e alla deserializzazione ricorsiva dei suoi elementi
28 strutturali.
- 29 • **StructuralElement**: Una classe base polimorfica specializzata nelle sottoclassi
30 `Title`, `Chapter` e `Article`. Permette di mappare i livelli intermedi di aggregazione
31 del testo, mantenendo traccia del numero d'ordine e del titolo della sezione.
- 32 • **Paragraph**: Rappresenta l'unità atomica informativa e testuale, ovvero il singolo
33 comma o blocco di testo non ulteriormente scindibile, sul quale verranno poi calcolati
34 gli embedding.

1 3.3.2 Disaccoppiamento della Persistenza: Lo Strategy Pattern

2 L'esigenza di effettuare una doppia ingestion prima su Neo4j e poi su Qdrant senza dover
3 duplicare la logica di orchestrazione o modificare il flusso principale del programma ha
4 reso naturale l'adozione del design pattern *Strategy*. Questo pattern comportamentale
5 definisce una famiglia di algoritmi, ne incapsula ciascuno all'interno di una classe e li
6 rende tra loro intercambiabili, consentendo all'algoritmo di variare indipendentemente
7 dai client che lo utilizzano.

8 Nel sistema sviluppato, l'interfaccia astratta di riferimento è rappresentata dalla classe
9 DBImporterStrategy. Essa definisce tre metodi fondamentali che ogni classe concreta
10 deve implementare:

- 11 • `connect()`: per la gestione e la validazione della connessione con l'istanza specifica
12 del database;
- 13 • `import_legal_act(entity, legal_act)`: per l'esecuzione materiale della logi-
14 ca di Ingestion dell'atto normativo e dei metadati dell'ente emittente;
- 15 • `close()`: per la chiusura sicura delle sessioni.

16 Nello script principale `main.py`, il ciclo di esecuzione batch istanzia dinamicamente
17 l'oggetto concreto (`Neo4jImporter` o `QdrantImporter`) a seconda della configurazione
18 desiderata. Questa architettura estendibile riduce a zero l'accoppiamento e semplifica
19 future implementazioni con ulteriori database.

20 3.3.3 Modalità di Accesso ai Dati: Il Repository Pattern

21 Se lo *Strategy Pattern* isola i database nella loro interezza rispetto all'orchestrazione
22 principale, il *Repository Pattern* interviene specificamente all'interno dell'architettura di
23 persistenza dei database per mediare tra lo strato dei modelli di dominio e lo strato di
24 accesso ai dati fisici. L'obiettivo principale del Repository Pattern è centralizzare la logica
25 di costruzione ed esecuzione delle query, offrendo al client un'interfaccia astratta per poter
26 salvare i dati nei database. In questo modo, l'importer concreto delega l'interazione fisica
27 con il database ai singoli repository, i quali ricevono l'oggetto di dominio da persistere,
28 nascondendo completamente i dettagli delle implementazioni delle query.

29 3.3.4 Dettagli Implementativi dell'Ingestion su Neo4j

30 Nel contesto di Neo4j, un atto legale non può essere salvato tramite un'unica operazione
31 atomica standard, a causa delle profonde interconnessioni tra i nodi della gerarchia.
32 Aniché concentrare la scrittura di stringhe Cypher complesse all'interno della classe
33 `Neo4jImporter`, la responsabilità è stata distribuita su una serie di repository specializzati,
34 ognuno deputato alla gestione del ciclo di vita di un singolo modello di dominio:

- 35 • `EntityRepository`: Gestisce l'inserimento dei nodi relativi agli enti istituzionali.

- 1 • **LegalActRepository**: Inserisce il nodo radice dell'atto e lo ancora all'ente emit-
- 2 tente.
- 3 • **StructuralElementRepository**: Incapsula la logica Cypher necessaria a creare i
- 4 nodi intermedi del grafo e a tessere gli archi gerarchici verso i rispettivi nodi padri.
- 5 • **ParagraphRepository**: Si occupa esclusivamente della persistenza dei nodi foglia
- 6 contenenti il testo dei commi.

7 Per quanto riguarda il salvataggio delle relazioni trasversali (ad esempio, le citazioni
 8 tra diversi atti normativi), queste informazioni vengono lette da file JSON esterni dedicati.
 9 Il metodo che si occupa di questa operazione attraverso i `custom_id` dei nodi salvati e
 10 indicizzati sul grafo trova in maniera quasi istantanea i nodi correlati dall'estratto JSON
 11 ed esegue una query Cypher dedicata procedendo alla creazione dell'arco semantico.

Listing 3.1 Esempio di relazione CITES estratta in formato JSON

```

12 1: {
13 2:   "sourceId": "ente-provincia-taranto__CCNL-22-02-2010__article
14   -3__paragraph-1",
15 3:   "relationshipType": "CITES",
16 4:   "targetId": "ente-provincia-taranto__CCNL-22-02-2010__article
17   -2"
18 5: }
```

20 3.3.5 Dettagli Implementativi dell'Ingestion su Qdrant

21 La strategia implementata nella classe `QdrantImporter` risponde a logiche radicalmente
 22 differenti rispetto a quelle dei database a grafo, dettate dai vincoli operativi dei database
 23 vettoriali e dalle necessità algoritmiche della ricerca semantica. Qdrant, infatti, non
 24 supporta nativamente strutture gerarchiche ramificate, ma organizza le informazioni in
 25 collezioni di vettori piatti denominati *Points*.

26 Il fulcro del processo di ingestion in Qdrant risiede nell'operazione di appiattimento
 27 (*flattening*) della struttura ad albero dell'atto normativo, la quale è stata configurata
 28 secondo tre fasi logiche:

- 29 • **Identificazione dell'unità fondamentale**: L'unità minima di indicizzazione vie-
- 30 ne individuata nel modello Paragraph (il singolo comma). Questa porzione di testo
- 31 rappresenta il compromesso ottimale per bilanciare la granularità informativa e la
- 32 focalizzazione semantica del modello di embedding.
- 33 • **Il problema del contesto assoluto**: Memorizzare un comma in modo isolato
- 34 causerebbe una perdita critica di contesto, rendendo impossibile determinare a
- 35 quale articolo o sezione appartenga il testo recuperato.
- 36 • **Risoluzione tramite *Tree-Climbing***: Per preservare l'integrità strutturale, l'al-
- 37 goritmo esegue una procedura di risalita in memoria per ciascun comma. Partendo
- 38 dall'elemento foglia (il comma), risale ricorsivamente la catena dei nodi padri,

1 estraendo il numero e il titolo di ogni livello gerarchico attraversato (Articolo, Capo,
2 Titolo).

3 Tutte le informazioni così raccolte vengono infine aggregate e inserite all'interno del
4 dizionario di metadati del punto vettoriale, denominato **Payload**

Listing 3.2 Esempio reale di un Point vettoriale su Qdrant con Payload

```

5 1: {
6 2:   "id": "00020dff-edff-51b8-b2fb-a953ed7e5c0e",
7 3:   "payload": {
8 4:     "p_id": "ente-provincia-taranto__regolamento-area-delle-
9       posizioni-organizzative-e-delle-alte-
10      professionalita__chapter-2__article-14__paragraph-1",
11 5:     "text": "Il presente Regolamento entrera in vigore ad
12      avvenuta pubblicazione del medesimo all'Albo Pretorio on-
13      line nonche sul sito web ufficiale dell'Ente.",
14 6:     "paragraph_num": "1",
15 7:     "act_name": "Regolamento Area delle Posizioni Organizzative
16      e delle Alte Professionalita.json",
17 8:     "article_num": "14",
18 9:     "article_name": "Entrata in vigore",
19 10:    "chapter_num": "2",
20 11:    "chapter_name": "Area delle Alte Professionalita"
21 12:  },
22 13:  "vector": [/* Rappresentazione numerica embedding testuale */]
23 14: }

```

25 AGGIUNGERE TEMPI DI INGESTION E NUMERO DI RECORD INSERITI PER OGNI
26 DATABASE

27 3.4 Creazione degli Embedding

28 L'operazione di vettorizzazione, o di *embedding*, rappresenta un passaggio cruciale per
29 abilitare le funzionalità di ricerca semantica sui documenti legali all'interno del sistema.

30 3.4.1 Astrazione e Gestione dei Modelli di Embedding

31 Per garantire la massima flessibilità nell'uso di modelli eterogenei, è stata definita un'inter-
32 faccia astratta base denominata `EmbeddingModel`. Da essa derivano le implementazioni
33 specifiche per le diverse tipologie di erogazione dei modelli:

- 34 • **Modelli Locali (`LocalEmbeddingModel`):** consentono l'esecuzione di modelli
35 open-source direttamente in locale sfruttando la libreria `SentenceTransformer`. I
36 modelli vengono salvati in una cache locale per ottimizzare i caricamenti successivi.
- 37 • **Modelli Cloud HuggingFace (`HuggingFaceCloudModel`):** si interfacciano con
38 la *HuggingFace Inference API*. Per garantire la resilienza del sistema, la classe imple-
39 menta un meccanismo di tolleranza ai guasti basato su tentativi multipli in caso di

- 1 errore di rete o di superamento dei limiti di utilizzo, con attese esponenziali tra i
2 tentativi.
- 3 • **Modelli Cloud Gemini (GeminiCloudModel):** comunicano tramite l'endpoint
4 REST embedContent di Google. Anche qui è stato implementato il meccanismo di
5 retry e, inoltre, per ottimizzare ulteriormente il throughput, viene utilizzata una
6 strategia di rotazione circolare su un pool di API keys fornite in configurazione.
- 7 In particolare, sono stati integrati quattro specifici modelli di embedding per verificare le
8 differenze prestazionali e di accuratezza in fase di retrieval:
- 9 • **google/embeddinggemma-300m:** Si tratta dell'unico modello della suite eseguito
10 interamente in locale sfruttando le risorse hardware della macchina tramite la
11 libreria SentenceTransformer. Produce vettori con una dimensionalità di 768.
- 12 • **intfloat/multilingual-e5-large:** Modello ad alte prestazioni fruibile in cloud
13 tramite le *HuggingFace Inference API*. Caratterizzato da una dimensionalità di 1024
14 vettori, questo modello richiede obbligatoriamente un pre-processing del testo
15 tramite prefissi asimmetrici per ottimizzare le prestazioni di recupero: nello specifico,
16 viene anteposto il token "passage: " per i testi da memorizzare e il token "query: "
17 per le stringhe di ricerca.
- 18 • **BAAI/bge-m3:** Modello cloud multilingua integrato anch'esso tramite l'infrastrut-
19 tura di *HuggingFace*. Genera vettori a 1024 dimensioni ed è particolarmente rino-
20 mato per la sua versatilità nel mappare relazioni semantiche complesse in contesti
21 cross-lingua e multi-lingua.
- 22 • **gemini/gemini-embedding-001:** Modello cloud proprietario di Google acces-
23 sibile mediante l'endpoint REST embedContent. Configurato con vettori a 768
24 dimensioni, è nativamente ottimizzato per task di *text retrieval* su larga scala.

25 3.4.2 Integrazione con i Database

26 La distribuzione dei vettori generati ai diversi database viene gestita, anche qui, attraverso
27 lo *Strategy Pattern*, che isola le logiche di inserimento dei dati per ciascun database.

28 Un aspetto cruciale condiviso da entrambe le strategie è l'**arricchimento contestuale**
29 **del testo**. Al fine di massimizzare l'accuratezza della ricerca semantica, il sistema non si
30 limita a generare l'embedding del solo testo grezzo del comma. Al contrario, il testo viene
31 dinamicamente concatenato con i suoi metadati gerarchici e descrittivi (come il nome
32 dell'atto normativo, il numero e il titolo del capitolo, e il numero e la rubrica dell'articolo).

33 La Strategia Neo4j

34 La Neo4jEmbeddingStrategy gestisce l'estrazione, la vettorizzazione e il salvataggio
35 degli embedding nel database a grafo. Il processo si articola in tre fasi principali:

- 1 **1. Estrazione e arricchimento contestuale:** Il sistema esegue una query Cypher
2 che risale la gerarchia del documento a partire dal testo del nodo (`:Paragraph`). La
3 query estrae i metadati dei nodi padre (Atto, Titolo, Capo, Articolo) e li concatena
4 al contenuto del comma, generando una stringa arricchita e contestualizzata.
- 5 **2. Generazione degli embedding:** Il testo completo viene inviato ai modelli di
6 *embedding* che processano e traducono il testo in vettori numerici in grado di
7 catturarne il significato semantico.
- 8 **3. Salvataggio nel database:** Infine, i vettori generati vengono salvati nel grafo. Vie-
9 ne creato un nuovo nodo dedicato assegnandogli due *label*: una generica (`:Embedding`)
10 e una dinamica, specifica per il modello utilizzato (es. `:Embedding_embeddinggemma_300m`).
11 Questo nodo memorizza il vettore generato e i dettagli del modello. Il proces-
12 so si conclude creando una relazione [`:HAS_EMBEDDING`] che connette il nodo
13 (`:Paragraph`) originario al suo rispettivo nodo (`:Embedding`).

14 **La Strategia Qdrant**

15 La classe `QdrantEmbeddingStrategy` verifica a runtime lo schema della collezione, ini-
16 zializzandone, ove necessario, la configurazione per i *Named Vectors*. Tale approccio
17 consente di associare a un singolo record vettoriale molteplici rappresentazioni prodotte
18 dai modelli differenti, agevolando le prestazioni in fase di retrieval.

19 In linea con l'approccio adottato per Neo4j, il processo implementa un pre-processing
20 di arricchimento semantico. Prima della generazione dell'embedding, vengono estratti
21 i metadati dal *payload* del database per costruire una gerarchia di contesto testuale (es.
22 "*Atto: Codice Civile | Titolo 1 | Capo 2 | Art. 4*").

1

Elenco delle figure

2	2.1	Architettura del sistema	5
3	3.1	Flowchart del Custom Triage Processor	10

Elenco delle tabelle